

UNIT-4 User Interface Design

User interface Design

- o The visual part of a computer application or operating system through which a client interacts with a computer or software.
- o It determines how commands are given to the computer or the program and how data is displayed on the screen.

o Types of User Interface:

There are two main types of User Interface:

1. Text-Based User Interface or Command Line Interface
2. Graphical User Interface (GUI)

User Interface Golden rules

1. Place the user in control:

- o The user should be able to easily enter and exit the mode with little or no effort.
 - o Provide for flexible interaction using keyboard, mouse or touch screen etc.
 - o Allow user interaction to be interruptible and undoable to do some other work.
2. **Strive for consistency** - Consistent sequences of actions should be required in similar situations. Identical terminology should be used in prompts, menus, and help screens. Consistent commands should be employed throughout.
- o Allow the user to put the current task into a meaningful context.
 - o Maintain consistency across a family of applications.
 - o Keep interaction results the same as per user expectations.

3. **Enable frequent users to use short-cuts** - The user's desire to reduce the number of interactions increases with the frequency of use. Abbreviations, function keys, hidden commands, and macro facilities are very helpful to an expert user.

4. **Offer informative feedback** - For every operator action, there should be some system feedback. For frequent and minor actions, the response must be modest, while for infrequent and major actions, the response must be more substantial.

5. **Design dialog to yield closure** - Sequences of actions should be organized into groups with a beginning, middle, and end. The informative feedback at the completion of a group of actions gives the operators the satisfaction of accomplishment, a sense of relief, the signal to drop contingency plans and options from their minds, and this indicates that the way ahead is clear to prepare for the next group of actions.

- o Sequences of actions should be organized into groups with a beginning, middle, and end.

- o This indicates that the way ahead is clear to prepare for the next group of actions.

6. Offer simple error handling - As much as possible, design the system so the user will not make a serious error. If an error is made, the system should be able to detect it and offer simple, comprehensible mechanisms for handling the error.

7. Permit easy reversal of actions - This feature relieves anxiety, since the user knows that errors can be undone. Easy reversal of actions encourages exploration of unfamiliar options. The units of reversibility may be a single action, a data entry, or a complete group of actions.

8. Support internal locus of control - Experienced operators strongly desire the sense that they are in charge of the system and that the system responds to their actions. Design the system to make users the initiators of actions rather than the responders.

9.Reduce short-term memory load - The limitation of human information processing in short-term memory requires the displays to be kept simple, multiple page displays be consolidated, window-motion frequency be reduced, and sufficient training time be allotted for codes, mnemonics, and sequences of actions.

- o Reduce demand on short-term memory. remembering of previously done actions, given inputs and results.

- o New features updated in system. o Define shortcut keys.

- o The visual layout of the interface

User Interface Analysis and Design models and process.

User Interface Analysis

- o The analysis and design process of a user interface is iterative and can be represented by a spiral model.

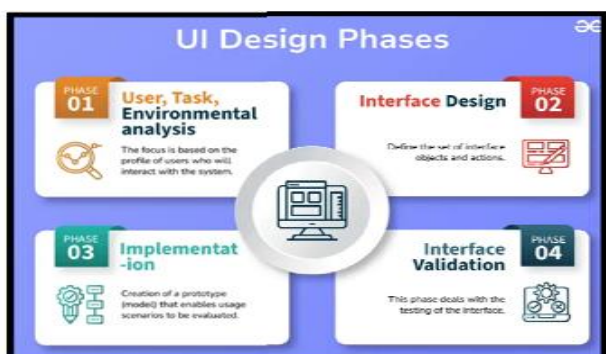
- o The analysis and design process of user interface consists of four framework activities.

1. User, Task, Environmental Analysis, and Modeling

2. Interface Design

3. Interface Construction and Implementation

4. Interface Validation



1. User, Task, Environmental Analysis, and Modeling:

- o Initially, the focus is based on the profile of users who will interact with the system, i.e., understanding, skill and knowledge, type of user, etc.
- o Based on the user's profile users are made into categories. From each category requirements are gathered.
- o Based on the requirement's developer understand how to develop the interface.

2. Interface Design:

- o The goal of this phase is to define the set of interface objects and actions i.e., control mechanisms that enable the user to perform desired tasks.
- o Indicate how these control mechanisms affect the system. Specify the action sequence of tasks and subtasks, also called a user scenario

3.Interface Construction and Implementation:

- o The implementation activity begins with the creation of a prototype (model) that enables usage scenarios to be evaluated.
- o As iterative design process continues a User Interface toolkit that allows the creation of windows, menus, device interaction, error messages, commands, and many other elements of an interactive environment can be used for completing the construction of an interface.

4.Interface validation:

- o This phase focuses on testing the interface.
- o The interface should be in such a way that it should be able to perform tasks correctly, and it should be able to handle a variety of tasks. It should achieve all the user's requirements.
- o It should be easy to use and easy to learn. o Users should accept the interface as a useful one in their work.

5.. Collaboration with Developers: Work closely with developers to ensure the UI design is feasible and can be implemented effectively. Use design systems or component libraries for consistency in implementation.

6. Usability Testing: Conduct usability testing throughout the development process to identify and address user experience issues. Use feedback to refine and improve the UI design.

7. Performance Considerations: Optimize the UI for performance to ensure smooth interactions and response times .Consider factors like page load times and responsiveness.

8. Security: Ensure that the UI design aligns with security best practices.

User Interface Design Process:

1.Understand the Users and Context: User Research: Conduct surveys, interviews, and observations to understand the target audience and their needs. Contextual Inquiry: Gain insights by observing users in their natural environment.

2.Define the Requirements: User Personas: Create fictional characters representing different user types. User Stories: Define specific, concrete use cases from the user's perspective.

3.Design the Information Architecture: Card Sorting: Organize content and features into categories based on user input. Sitemaps: Create a hierarchical diagram representing the structure of the website or application.

4.Create Wireframes: Low-Fidelity Wireframes: Basic sketches to outline the structure and layout. High-Fidelity Wireframes: Detailed representations, often created using design tools.

5.Prototype: Interactive Prototypes: Build a clickable model of the design for user testing. Usability Testing: Gather feedback on the prototype to identify potential issues.

6.Visual Design: Style Guides: Define the visual elements, such as colors, fonts, and icons. Mockups: Create high-fidelity representations of the final product.

7.Development and Implementation: Collaboration with Developers: Work closely with developers to ensure the design is implemented accurately. Iterative Testing: Test the UI during the development phase for any issues

Web App and mobile interface design (interface design principles and guidelines)

Web App Interface Design:

1. Responsive Design: Ensure a smooth user experience across various devices and screen sizes.

2. Simple Navigation: Use clear and consistent navigation menus and buttons.

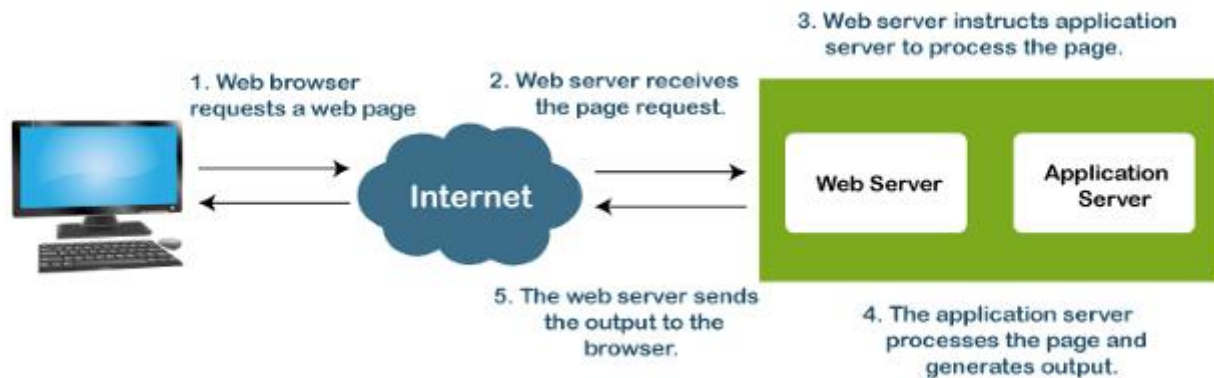
3. Prominent Calls-to-Action: Use eye-catching CTAs to guide users through the app.

4. Intuitive Information Architecture: Organize content in a logical and easy-to-follow manner.

5. Accessibility: Design for users with disabilities by following WCAG guidelines.

Web Application: A web-application is an application program that is usually stored on a remote server, and users can access it through the use of Software known as web-browser.

The Flow of the Web Application



1. In general, a user sends a request to the web-server using web browsers such as Google Chrome, Microsoft Edge, Firefox, etc over the internet.
2. Then, the request is forwarded to the appropriate web application server by the web-server.
3. Web application server performs the requested operations/ tasks like processing the database, querying the databases; produces the result of the requested data.
4. The obtained result is sent to the web-server by the web application server along with the requested data/information or processed data.
5. The web server responds to the user with the requested or processed data/information and provides the result to the user's screen.

Mobile interface design

1. Mobile interface design refers to the process of creating visual and interactive elements for applications and websites that are specifically tailored for mobile devices such as smartphones and tablets.
2. It encompasses the design of the user interface (UI) and user experience (UX) to ensure that users can interact with the application or website seamlessly on smaller screens.
3. Thumb-Friendly Design: Ensure easy navigation and interaction with a thumb-friendly layout.
4. Simple and Concise Content: Use clear and concise language, avoiding clutter and unnecessary elements.
5. Prominent Calls-to-Action: Use large, tap friendly CTAs to guide users through the app.
6. Mobile-Specific Features: Leverage mobile specific features like GPS, camera, and push notifications.
7. Fast and Responsive Performance: Optimize performance for seamless interactions and fast loading times.

Interface design principles:

- 1. Clarity and Simplicity:** Keep the interface simple and easy to understand. Avoid unnecessary elements that may clutter the screen. Prioritize clarity in design and communication.
- 2. Consistency:** Consistency is important in user interface design, as it helps users to understand and learn how to use an application. Consistent design elements such as icons, color schemes, and navigation menus should be used throughout the application.
- 3. Simplicity:** User interfaces should be designed to be simple and easy to use, with clear and concise language and intuitive navigation.
- 4. Feedback:** Feedback is significant in user interface design, Feedback can take the form of visual cues, messages, or sounds.
- 5. Accessibility:** User interfaces should be designed to be accessible to all users, regardless of their abilities. This involves considering factors such as color contrast, font size, and assistive technologies such as screen readers.
- 6. Flexibility:** User interfaces should be designed to be flexible and customizable.
- 7. Intuitive Navigation:** Design navigation in a way that feels natural and intuitive to users. Mobile interfaces often rely on gestures, so ensure that users can easily navigate through the app with common gestures like swiping and tapping.
- 8. Thumb-Friendly Design:** Place interactive elements within easy reach of the user's thumb. Users should be able to navigate and interact with the app comfortably using one hand.
- 7. Responsive Design:** Ensure that the interface adapts seamlessly to different screen sizes and orientations. A responsive design provides a consistent experience across a variety of devices.
- 8. Feedback and Affordance:** Provide clear feedback for user actions. Use visual and audio cues to indicate that a button has been pressed or an action has been completed. Affordance refers to making elements look like they can be interacted with, making the interface more user-friendly.
- 9. Minimize User Input:** Reduce the number of steps and input fields required for users to complete tasks. Simplify forms and utilize input options such as auto-complete and input masks.
- 10. Performance Optimization:** Optimize the performance of the application to ensure fast loading times and smooth interactions. Slow-loading content can frustrate users and lead to a poor experience.
- 11. Readable Typography:** Choose legible fonts and appropriate text sizes. Ensure that text is easily readable on various screen sizes and resolutions.
- 12. Touch-Friendly Buttons:** Design interactive elements, such as buttons, with an appropriate size to facilitate easy tapping. Provide enough spacing between elements to prevent accidental taps.
- 13. User Testing:** Conduct user testing to gather feedback on the usability of the interface. Real user feedback can reveal insights into how users interact with the app and help identify areas for improvement.

Interface design guidelines:

1. Layout:

- o Grid System: Use a grid to organize elements and create balance
- o White Space: Use sufficient white space to create a clean and uncluttered design³
- o Grouping: Group related elements together

2. Typography:

- o Clear Hierarchy: Use headings, subheadings, and body text to create a clear hierarchy
- o Consistent Font: Use a consistent font throughout the design
- o Readable Font Size: Ensure font sizes are readable on various devices

3. Color:

- o Contrast: Ensure sufficient contrast between text and background
- o Consistency: Use a consistent color scheme throughout the design
- o Accessibility: Consider color blindness and use accessible colors.

4. Icons and Graphics:

- o Simple and Consistent: Use simple and consistent icons and graphics
- o Scalable: Use scalable graphics to ensure they look good on various devices
- o Meaningful: Use icons and graphics that are meaningful and intuitive

Implementation issues: Reuse, configuration management, Host target development

Reuse

1.Code Reusability:

Issue: Difficulty in identifying and reusing existing code or components.

Solution: Implement modular and well-documented code. Utilize design patterns and frameworks to encapsulate reusable functionality.

2.Library and Component Management:

Issue: Managing external libraries and components can lead to versioning conflicts or dependency issues.

Solution: Use package managers or dependency management tools. Keep libraries up-to date and adhere to versioning conventions.

3.Knowledge Transfer:

Issue: Lack of knowledge transfer and documentation on reusable components.

Solution: Document code thoroughly. Establish a knowledge-sharing culture within the development team. Use code reviews and pair programming.

4.Design for Reusability:

Issue: Components not designed with reusability in mind.

Solution: Encapsulate functionalities into modular components. Follow SOLID principles. Strive for a balance between generality and specificity.

5. Testing Reusable Components:

Issue: Ensuring that reused components work correctly in different contexts.

Solution: Implement thorough unit testing for reusable components. Conduct integration tests to validate the compatibility of reused code.

Configuration Management:

1. Version Control:

Issue: Difficulty in managing and tracking changes to the codebase.

Solution: Use version control systems (e.g., Git, SVN) to track changes, manage branches, and collaborate efficiently. Establish branching and merging strategies .

2. Build and Deployment Configuration:

Issue: Inconsistencies in build and deployment configurations.

Solution: Use configuration files to manage build settings. Automate build and deployment processes. Utilize containerization technologies like Docker for consistent deployment environments.

3. Environment Configuration:

Issue: Configurations that are hard-coded, leading to inflexibility in different environments.

Solution: Use configuration files or environment variables to store environment-specific settings. Implement a configuration management system.

4. Dependency Management:

Issue: Unmanaged dependencies causing conflicts or security vulnerabilities.

Solution: Regularly update dependencies. Use tools like package managers to manage dependencies. Conduct regular security audits.

5. Documentation:

Issue: Inadequate documentation of configurations and changes.

Solution: Maintain detailed documentation for configuration settings, changes, and deployment processes. Ensure that documentation is regularly updated.

6. Rollback Mechanism:

Issue: Lack of a reliable mechanism to roll back changes in case of issues.

Solution: Implement a rollback strategy. Maintain backups and versioned releases to facilitate rollback if needed.

Host-Target Development:

- o **Cross-compilation:** Use cross-compilation tools to compile code for target platforms.
- o **Emulation:** Use emulation tools to test and debug code on host platforms.
- o **Remote debugging:** Use remote debugging tools to debug code running on target platforms.
- o **Boot loaders and firmware:** Develop boot loaders and firmware for target platforms.
- o **Platform-specific optimization:** Optimize code for specific target platforms.

Software quality: Software quality refers to the degree to which a software system meets its specified requirements, is reliable, efficient, and maintainable, and satisfies the needs of its users.

- Software quality product is defined in term of its fitness of purpose. That is, a quality product does precisely what the users want it to do. For software products, the fitness of use is generally explained in terms of satisfaction of the requirements laid down in the SRS document. Although "fitness of purpose" is a satisfactory interpretation of quality for many devices such as a car, a table fan, a grinding machine, etc. for software products, "fitness of purpose" is not a wholly satisfactory definition of quality



Example: Consider a functionally correct software product. That is, it performs all tasks as specified in the SRS document. But, has an almost unusable user interface. Even though it may be functionally right, we cannot consider it to be a quality product.

The modern view of a quality associated with a software product several quality methods such as the following:

- **Portability:** A software device is said to be portable, if it can be freely made to work in various operating system environments, in multiple machines, with other software products, etc.
- **Usability:** A software product has better usability if various categories of users can easily invoke the functions of the product. **Reusability:** A software product has excellent reusability if different modules of the product can quickly be reused to develop new products.
- **Correctness:** A software product is correct if various requirements as specified in the SRS document have been correctly implemented.
- **Maintainability:** A software product is maintainable if bugs can be easily corrected as and when they show up, new tasks can be easily added to the product, and the functionalities of the product can be easily modified, etc.

McCall's quality factors

McCall's quality factors, introduced by John McCall in the 1970s, are a set of software quality attributes that serve as a framework for evaluating and measuring the quality of a software product. These factors help identify and define various aspects of software quality.

McCall's model consists of three main categories: product revision, product transition, and product operation. Within these categories, there are eleven quality factors

1. Product Revision Factors:

- 1. Correctness:** The degree to which the software product meets its specifications and fulfills the user's requirements without errors.
- 2. Reliability:** The ability of the software to perform its intended functions without failure over a specified period.
- 3. Efficiency:** The ability of the software to perform its functions with minimal resource utilization, such as processing time and memory usage.
- 4. Integrity:** The degree to which the software safeguards against unauthorized access or alterations of its data.
- 5. Usability:** The ease with which users can learn, interact with, and use the software to achieve their goals.

2. Product Transition Factors:

1.Portability:The ease with which the software can be transferred from one environment or platform to another.

2.Reusability:The extent to which software components can be reused in different applications or parts of the same application.

3.Interoperability:The ability of the software to interact and operate with other specified software systems.

3. Product Operation Factors:

1.Maintainability:The ease with which the software can be modified or enhanced to adapt to changing requirements or correct defects.

2.Flexibility: The degree to which the software can be adapted or extended to accommodate future changes in requirements.

3.Testability:The ease with which the software can be tested to ensure that it meets its specified requirements.

ISO 9126 quality factors

- ISO/IEC 9126 is an international standard for software product quality. The standard defines a set of characteristics and sub-characteristics that can be used to evaluate and measure the quality of software.
- ISO/IEC 9126 is structured into three main categories: product quality, external quality, and internal quality. Within these categories, there are six primary quality characteristics, each with sub-characteristics.

The six primary quality characteristics are:

1.Functionality:The degree to which the software provides functions that meet stated and implied needs when used under specified conditions.

Sub-characteristics:

- Suitability
- Accuracy
- Interoperability
- Compliance
- Security

2. **Reliability:**The ability of the software to maintain a specified level of performance when used under specified conditions.

Sub-characteristics:

- Maturity
- Fault Tolerance
- Recoverability
- Reliability

3. **Usability:**The extent to which the software can be understood, learned, used, and attractive to the user when used under specified conditions.

Sub-characteristics:

- Understandability
 - Learnability
 - Operability
 - Attractiveness
3. **Efficiency:** The degree to which the software uses resources in relation to the amount of work accomplished under specified conditions.

Sub-characteristics:

- Time Behavior
- Resource Utilization

- 5.**Maintainability:**The degree of effectiveness and efficiency with which the software can be modified to correct defects, modified, or adapted to changes in the environment.

Sub-characteristics:

- Analyzability
- Modifiability
- Stability
- Testability

- 6.**Portability:** The degree of effectiveness and efficiency with which the software can be transferred from one hardware, software, or operational environment to another.

Sub-characteristics:

- Adaptability
- INstallability
- Co-Existence
- Replaceability

